Enterprise AI Network Intelligence Platform

Implementation Plan

Build an Enterprise AI Network Intelligence Platform with Kafka-backed high-volume simulated KPI ingestion, Isolation Forest + LSTM anomaly detection, forecasting, RAG root-cause analysis, MLflow, and local Kubernetes (kind/k3d) first ? Helm charts designed for later cloud promotion.

1. Locked Decisions

- Data: High-volume simulated network KPIs; ingestion via Kafka (Redpanda for local simplicity, Kafka-compatible).
- Runtime: Local Kubernetes first (kind or k3d); Helm charts reusable for EKS/GKE/AKS later.
- LLM (default): OpenAI API for RAG RCA; swap to Azure OpenAI/Ollama via config.
- Dashboards: Custom React executive dashboard + Grafana for ops/ML metrics.

2. Architecture Overview

Simulator -> Redpanda/Kafka -> Ingest Workers -> TimescaleDB. TimescaleDB feeds Anomaly Service, Forecast Service, and RAG RCA. Anomalies drive Alert Router (Slack/Email). MLflow tracks experiments; retrain pipeline promotes models. Prometheus/Grafana monitor the platform. API Gateway serves the Executive Dashboard.

3. Core Data Flow

1. Simulator emits multi-device KPIs (latency, packet loss, CPU, interface util, error rates) at high rate into Kafka topics.
2. Ingest workers batch-consume, validate, write to TimescaleDB, expose Prometheus metrics.
3. Online anomaly (Isolation Forest for multivariate point anomalies; LSTM for sequential) and forecasting services read recent windows from TSDB.
4. On anomaly/forecast breach -> Alert Router (email + Slack) + RAG RCA (retrieve runbooks/topology + LLM explanation).
5. MLflow tracks experiments; scheduled retrain job pulls labeled/windowed data, logs models, promotes via registry for K8s rollouts.

4. What is Redpanda?

Redpanda is a streaming platform that speaks the Kafka protocol. Producers and consumers use the same Kafka client libraries, but Redpanda is a single binary (no ZooKeeper/KRaft ops overhead), which makes local Kubernetes setup much simpler than Apache Kafka.

- Redpanda = the message bus / buffer between the simulator and the database.
- It holds recent KPI events in topics until ingest workers catch up.
- It is NOT the long-term analytics store.
- Later you can swap Redpanda for Amazon MSK / Confluent Cloud without rewriting producers/consumers.

5. How Data Injection (Simulation) Works

We do not pull from real routers in v1. A Python service kpi-simulator generates fake but realistic network telemetry and publishes it to Kafka/Redpanda.

- Config: number of devices, metrics per device, target messages/sec, fault injection

rate.
- Each message: { "ts", "device_id", "site", "metric", "value" }.
- Normal traffic: baselines + noise (sine/diurnal patterns).
- Fault injection: periodically spike latency, packet loss, or CPU for anomaly detection.
- Partition key = device_id (ordered per device, parallel across devices).
- Why Kafka in front of DB: absorbs bursts; workers batch-write at a sustainable rate.

6. Where Data is Stored

- In-flight KPI events: Redpanda topics (kpis.raw, kpis.anomalies) ? buffer/stream.
- Historical KPI time series: TimescaleDB hypertable kpi_samples ? source of truth.
- Dashboard rollups: TimescaleDB continuous aggregates ? fast executive UI queries.
- Anomaly / forecast results: TimescaleDB tables (anomalies, forecasts).
- Runbooks / topology for RAG: pgvector (same Postgres) or Chroma.
- Trained models + artifacts: MLflow metadata DB + MinIO object storage.
- Ops metrics: Prometheus ? Grafana monitoring only (separate from network KPIs).

Mental model: Simulator -> Redpanda (pipe) -> TimescaleDB (warehouse) -> ML + Dashboard.
Prometheus/Grafana watch the platform itself; they do not store business KPI history.

7. Requirements / Setup

Machine / tooling

- Docker Desktop (or Docker Engine) with 16 GB+ RAM (32 GB ideal).
- kubectl, Helm 3, kind or k3d.
- Python 3.11+, Node.js 20+, Git.
- Optional GPU for LSTM training (CPU OK for v1 with smaller models).

Accounts / secrets

- OpenAI API key (or Ollama local endpoint).
- SMTP credentials (email alerts).
- Slack Incoming Webhook or Bot token.
- Optional: cloud registry later (ECR/GCR/ACR).

Local cluster

- kind/k3d cluster with ingress (nginx) and storage class.
- Namespaces: platform, ml, observability, streaming.

Runtime stack (v1)

- Streaming: Redpanda (Kafka API)
- KPI store: TimescaleDB (Postgres + hypertables)
- Vector DB: pgvector on same Postgres or Chroma
- API: FastAPI (Python)
- ML: scikit-learn (Isolation Forest), PyTorch (LSTM), Prophet/LSTM for forecast
- Tracking: MLflow + MinIO
- Orchestration: Argo Workflows or CronJob for v1
- Metrics: Prometheus + Alertmanager
- Viz: Grafana + React (Vite) executive UI
- Deploy: Helm charts, Docker images

8. Repository Layout

enterprise-netintel/

apps/kpi-simulator/
apps/ingest-worker/
apps/anomaly-service/
apps/forecast-service/
apps/rca-service/
apps/alert-router/
apps/api-gateway/
apps/executive-dashboard/
ml/training/
ml/pipelines/
infra/helm/
infra/kind/
infra/grafana/dashboards/
infra/prometheus/
docs/
docker-compose.dev.yml

9. Implementation Phases

- Phase 0 ? Bootstrap: monorepo, git, Makefile, .env.example, Dockerfiles, kind + Helm scaffolding.
- Phase 1 ? Streaming: Redpanda topics, simulator, ingest workers, TimescaleDB hypertables, Prometheus metrics.
- Phase 2 ? Anomaly + forecasting: Isolation Forest + LSTM, forecast service, MLflow logging, model registry.
- Phase 3 ? RAG RCA + alerting: runbook corpus, RCA service, Slack/email alert router, Alertmanager.
- Phase 4 ? Dashboards: React executive UI, Grafana ops dashboards, FastAPI gateway.
- Phase 5 ? Retraining: CronJob/Argo export -> train -> evaluate -> promote.
- Phase 6 ? Cloud readiness: same Helm charts, registry + CI, cloud secrets (later).

10. Design Choices for Large Simulated Data

- Kafka partitions aligned with device cardinality for parallel consumers.
- Batch writes (COPY / multi-row INSERT), compression on topics.
- Continuous aggregates instead of raw scans for executive UI.
- Separate online inference path from offline training path.
- Resource quotas in K8s so simulator cannot starve ML/API pods.

11. Success Criteria (v1 Demo)

- Sustained simulated ingest (tunable, e.g. 10k?100k msgs/sec depending on hardware).
- Anomalies visible in dashboard within seconds of fault injection.
- Forecast charts + Isolation Forest / LSTM scores.
- RCA paragraph + linked runbook snippets on alert.
- MLflow UI showing at least one train run; CronJob retrains successfully.
- Grafana + Prometheus scraping all services; Slack/email on injected anomaly.

12. Out of Scope for v1

- Real SNMP/NetFlow adapters (interface stubs only).
- Multi-tenant RBAC / SSO (basic auth or open local).
- Full Kubeflow stack (MLflow + Argo is enough).
- Production HA multi-AZ cloud hardening.

13. Implementation Todos

1. Create enterprise-netintel monorepo, git, Docker/Helm/kind scaffolding, .env.example
2. Deploy Redpanda + TimescaleDB; build KPI simulator and ingest workers with Prometheus metrics
3. Implement anomaly (Isolation Forest + LSTM) and forecast services with MLflow tracking
4. Build RAG RCA service, runbook corpus, Slack/email alert router
5. React executive dashboard + Grafana dashboards + FastAPI gateway
6. Automatic retrain pipeline (CronJob/Argo) with model promotion
7. Umbrella Helm chart, deploy full stack on kind/k3d, document runbook